

DATA MODELLING - EXAMPLE - TESS PROJECT (TOI) CANDIDATES DATASET

**PURPOSE:** This notebook utilises TESS Projects Candidate data to detect short-period planets which may include sub-neptune candidates.

Cleaned data is balanced (SMOTE) and split before the following machine learning (ML)models are applied: Support Vector Classifier (SVC), Random Forest (RF), Light Gradient Boosting Machine (LGBM) and Multi-Layer Perceptron (MLP).

**INPUT:** TOI\_2025.03.30\_22.47.16\_cleaned.csv - TOI data as at March 2025

**OUTPUT:** Optimised Machine Learning models:

- model\_LGBM\_opt.joblib
- model\_MLP\_opt.joblib
- model\_RF\_opt.joblib
- model\_SVC\_opt.joblib

**PROCESSING:** Feature importance via Shapley Additive Data cleaning remove redundant features and instances, calculation of additional features and transformation.

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force\_remount=True).

```
# PYTHON LIBRARIES
import warnings # Display warning messages

warnings.filterwarnings('ignore')

import calendar      # Convert to names of month
import csv           # Reading in CSV files
import numpy as np    # Operations on arrays
import pandas as pd   # Data manipulation and analysis
import datetime as dt # Date and time manipulation
import math           # Mathematical functions
import seaborn as sns # Seaborn library
import os            # Directory

import matplotlib.pyplot as plt # Plots
import matplotlib.dates as mdate # Date manipulation for graphical display

from math import sqrt # SQRT

from scipy.stats import spearmanr # Spearman Correlation

import statsmodels.api as sm # Statsmodels library

# SKLEARN Library
from sklearn import datasets
from sklearn.model_selection import cross_val_predict      # ANN - Multilayer Perceptron model
from sklearn.model_selection import train_test_split      # Split method
from sklearn.model_selection import GridSearchCV          # Best parameter value
from sklearn.model_selection import cross_val_score       # Cross validation
from sklearn.metrics import classification_report, confusion_matrix # Reporting Metrics
from sklearn.metrics import accuracy_score              # Reporting Metrics

from sklearn.metrics import mean_squared_error          # RMSE
from sklearn.neural_network import MLPClassifier        # ANN Model
from sklearn.preprocessing import StandardScaler        # Standardises features by removing the mean and scaling to unit variance
from sklearn.pipeline import Pipeline                  # Pipeline Library

# Magic command to plot figures in the Notebook
%matplotlib inline

from IPython import get_ipython
get_ipython().magic('reset -sf')
```

# INSTALL PYSPARK

```
!pip install pyspark
```

Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)  
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)

# INSTALL IMBALANCED-LEARN (SMOTE)

```
!pip install imbalanced-learn
```

Requirement already satisfied: imbalanced-learn in /usr/local/lib/python3.12/dist-packages (0.14.1)  
Requirement already satisfied: numpy<3,>=1.25.2 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (2.0.2)  
Requirement already satisfied: scipy<2,>=1.11.4 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.16.3)  
Requirement already satisfied: scikit-learn<2,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.6.1)  
Requirement already satisfied: sklearn-compat<0.2,>=0.1.5 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (0.1.5)  
Requirement already satisfied: joblib<2,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.5.3)  
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (3.6.0)

# INSTALL SHAP

```
!pip install shap
```

Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.51.0)  
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)  
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from shap) (1.16.3)  
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)  
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)  
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)  
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (26.1)  
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)  
Requirement already satisfied: numba in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)  
Requirement already satisfied: llvmlite in /usr/local/lib/python3.12/dist-packages (from shap) (0.43.0)

Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)  
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)  
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2.9.0.post0)  
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)  
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2026.1)  
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (1.5.3)  
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.6.0)  
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)

## 1. RETRIEVE CLEANED TOI DATA

```
# READ CLEANED LOG TESS PLANETARY CANDIDATES FILE

# GET CLEANED, LOG EQUIVALENT FEATURES FILE

import pandas as pd # Data manipulation and analysis

df_TOI_cleaned = pd.read_csv('/content/drive/MyDrive/TOI_2025.03.30.csv')
print(df_TOI_cleaned.shape)
df_TOI_cleaned.head()
```

(2054, 25)

|   | toi    | tfopwg_disp | pl_tranmid  | st_tmag | st_logg | TESS<br>Mag | tfopwg_disp_encoded | ratio_eqt_teff | ratio_pl_orbper | ratio_pl_eqt | ... | log_st_teff | log_st_rad | log_Pred<br>(M_E |
|---|--------|-------------|-------------|---------|---------|-------------|---------------------|----------------|-----------------|--------------|-----|-------------|------------|------------------|
| 0 | 101.01 | KP          | 2458326.009 | 12.4069 | 4.48851 | 12.4069     | 1                   | 0.272483       | 9.219602        | 0.008642     | ... | 8.630700    | 0.636986   | 4.7              |
| 1 | 102.01 | KP          | 2460187.917 | 9.7109  | 4.32092 | 9.7109      | 1                   | 0.224204       | 3.506531        | 0.010988     | ... | 8.745284    | 0.792993   | 5.7              |
| 2 | 103.01 | KP          | 2458327.253 | 11.5232 | 4.22896 | 11.5232     | 1                   | 0.236948       | 4.110044        | 0.009690     | ... | 8.756525    | 0.875469   | 4.7              |
| 3 | 104.01 | KP          | 2460150.609 | 9.8638  | 3.93359 | 9.8638      | 1                   | 0.290768       | 3.333199        | 0.007762     | ... | 8.705662    | 1.168968   | 4.8              |
| 4 | 105.01 | KP          | 2459084.585 | 9.4995  | 4.37759 | 9.4995      | 1                   | 0.275212       | 6.272282        | 0.008844     | ... | 8.636042    | 0.805690   | 4.8              |

5 rows × 25 columns

df\_TOI\_cleaned.columns

```
Index(['toi', 'tfopwg_disp', 'pl_tranmid', 'st_tmag', 'st_logg', 'TESS Mag',  
      'tfopwg_disp_encoded', 'ratio_eqt_teff', 'ratio_pl_orbper',  
      'ratio_pl_eqt', 'st_mass_est', 'log_pl_rade', 'log_pl_eqt',  
      'log_pl_orbper', 'log_TSM', 'log_st_teff', 'log_st_rad',  
      'log_Predicted Mass (M_Earth)', 'log_pl_trandurh', 'log_pl_trandep',  
      'log_pl_insol', 'log_st_dist', 'log_ESM', 'log_ratio_pl_st_rad',  
      'log_ratio_rade_eqt'],  
      dtype='object')
```

### 1.2 - REMOVE NON-RELATED FEATURES

```
# DROP NON-RELEVANT FEATURE COLUMNS

df_TOI_cleaned_2 = df_TOI_cleaned.drop(columns=['toi', 'tfopwg_disp', 'pl_tranmid'])
print(df_TOI_cleaned_2.shape)
df_TOI_cleaned_2.head()
```

(2054, 22)

|   | st_tmag | st_logg | TESS<br>Mag | tfopwg_disp_encoded | ratio_eqt_teff | ratio_pl_orbper | ratio_pl_eqt | st_mass_est | log_pl_rade | log_pl_eqt | ...      | log_st_teff | log_st_rad | log      |
|---|---------|---------|-------------|---------------------|----------------|-----------------|--------------|-------------|-------------|------------|----------|-------------|------------|----------|
| 0 | 12.4069 | 4.48851 | 12.4069     |                     | 1              | 0.272483        | 9.219602     | 0.008642    | 0.891205    | 2.652358   | 7.330998 | ...         | 8.630700   | 0.636986 |
| 1 | 9.7109  | 4.32092 | 9.7109      |                     | 1              | 0.224204        | 3.506531     | 0.010988    | 1.117952    | 2.801577   | 7.250636 | ...         | 8.745284   | 0.792993 |
| 2 | 11.5232 | 4.22896 | 11.5232     |                     | 1              | 0.236948        | 4.110044     | 0.009690    | 1.211012    | 2.746106   | 7.317119 | ...         | 8.756525   | 0.875469 |
| 3 | 9.8638  | 3.93359 | 9.8638      |                     | 1              | 0.290768        | 3.333199     | 0.007762    | 1.540663    | 2.682649   | 7.470837 | ...         | 8.705662   | 1.168968 |
| 4 | 9.4995  | 4.37759 | 9.4995      |                     | 1              | 0.275212        | 6.272282     | 0.008844    | 1.333927    | 2.688042   | 7.346295 | ...         | 8.636042   | 0.805690 |

5 rows × 22 columns

#### 1.3.1 - SPLIT DATA AND CREATE LABELS

```
# BREAKUP TOI DATASET INTO FEATURES AND TARGET
# TARGET - DEPENDENT VARIABLE FOR PREDICTION - TFOPWG_DISP_ENCODED
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
# SEPARATE FEATURES AND TARGET VARIABLE
```

```
# TARGET VARIABLE (y)
y = df_TOI_cleaned_2['tfopwg_disp_encoded']
print(y.name)
```

```
# FEATURE MATRIX (X)
X = df_TOI_cleaned_2.drop('tfopwg_disp_encoded', axis=1)
```

```
print("Feature data dimension: ", X.shape, '\n')
```

```
y_cols = y.name
print(y_cols)
```

```
tfopwg_disp_encoded
Feature data dimension: (2054, 21)
```

```
tfopwg_disp_encoded
```

```
# SAVE SPLIT DATA TO FILE
```

```
print('save split data')
```

```
X.to_csv('/content/drive/MyDrive/TOI_2025.03.30_X.csv', index=False)
print("X data saved to TOI_2025.03.30_X.csv")

y.to_csv('/content/drive/MyDrive/TOI_2025.03.30_y.csv', index=False)
print("y data saved to TOI_2025.03.30_y.csv")
```

```
save split data
X data saved to TOI_2025.03.30_X.csv
y data saved to TOI_2025.03.30_y.csv
```

### 1.3.2 - RETRIEVE SPLIT DATA AND CREATE LABELS

```
# SPLIT THE DATASET INTO TRAINING AND TESTING SUBSETS
# TRAINING: TESTING 80/20 RATIO - AS PER PAPERS

from sklearn.model_selection import train_test_split

# SPLIT DATA INTO TRAIN, TEST
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print('X_train: ', X_train.shape)
print('X_test: ', X_test.shape)
print('y_train: ', y_train.shape)
print('y_test: ', y_test.shape)
```

```
X_train: (1643, 21)
X_test: (411, 21)
y_train: (1643,)
y_test: (411,)
```

```
# SAVE FEATURE NAMES FOR X_TRAIN/TEST
X_feature_names = X_train.columns
X_feature_names
```

```
Index(['st_tmag', 'st_logg', 'TESS Mag', 'ratio_eqt_teff', 'ratio_pl_orbper',
      'ratio_pl_eqt', 'st_mass_est', 'log_pl_rade', 'log_pl_eqt',
      'log_pl_orbper', 'log_TSM', 'log_st_teff', 'log_st_rad',
      'log_Predicted Mass (M_Earth)', 'log_pl_trandurh', 'log_pl_trandep',
      'log_pl_insol', 'log_st_dist', 'log_ESM', 'log_ratio_pl_st_rad',
      'log_ratio_rade_eqt'],
      dtype='object')
```

```
df_X_train = pd.DataFrame(X_train)
df_X_train.head()
```

|      | st_tmag  | st_logg | TESS<br>Mag | ratio_eqt_teff | ratio_pl_orbper | ratio_pl_eqt | st_mass_est | log_pl_rade | log_pl_eqt | log_pl_orbper | ... | log_st_teff | log_st_rad | log_r    |
|------|----------|---------|-------------|----------------|-----------------|--------------|-------------|-------------|------------|---------------|-----|-------------|------------|----------|
| 818  | 10.36400 | 3.74000 | 10.36400    | 0.085630       | 1.030121        | 0.031246     | 1.386257    | 2.900619    | 6.311735   | 2.872598      | ... | 8.767796    | 1.289233   | 0.000000 |
| 887  | 8.96760  | 4.33387 | 8.96760     | 0.285265       | 5.781712        | 0.003901     | 19.903907   | 2.452521    | 7.909489   | 1.042539      | ... | 9.163563    | 1.796747   | 0.000000 |
| 788  | 9.76952  | 4.43000 | 9.76952     | 0.431450       | 2.519016        | 0.000830     | 1.187730    | 1.095273    | 7.782807   | 0.582210      | ... | 8.623174    | 0.741937   | 0.000000 |
| 1530 | 11.64330 | 3.85000 | 11.64330    | 0.322691       | 4.708997        | 0.004290     | 1.562437    | 2.375334    | 7.729735   | 1.122160      | ... | 8.860499    | 1.241269   | 0.000000 |
| 619  | 8.50770  | 4.56000 | 8.50770     | 0.170265       | 1.082596        | 0.006533     | 0.764821    | 1.839203    | 6.698268   | 1.772883      | ... | 8.467646    | 0.565314   | 0.000000 |

5 rows × 21 columns

```
y_train.head()
```

```
tfopwg_disp_encoded
818      0
887      0
788      0
1530      0
619      1

dtype: int64
```

```
df_y_train = pd.DataFrame(y_train)
df_y_train.columns
```

```
Index(['tfopwg_disp_encoded'], dtype='object')
```

```
df_X_test = pd.DataFrame(X_test)
```

```
df_y_test = pd.DataFrame(y_test)
```

## 2.DATA ANALYSIS

Original data → Scaling → Correlation analysis → Feature pruning → SMOTE → Modeling

### 2.1 - APPLY SCALER

```
# FIRST SCALE THE FEATURES - X ONLY

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
print(X_train_scaled.shape)
print(X_test_scaled.shape)
print()

print(X_train_scaled)
print()
print(X_test_scaled)
```

```
(1643, 21)
(411, 21)

[[-3.64400733e-01 -1.84214551e+00 -3.64400733e-01 ... 9.95604641e-03
 -3.51550990e-04 1.93117103e+00]
 [-1.18692422e+00 6.83116879e-02 -1.18692422e+00 ... -3.49671485e-03
 -1.58981241e+00 -6.13797735e-01]
 [-7.14568136e-01 3.77558241e-01 -7.14568136e-01 ... -9.14334047e-01
 -1.80681114e+00 -2.50577028e+00]
 ...
 [-2.82702074e-01 -7.28164332e-02 -2.82702074e-01 ... -7.03374264e-01
 -1.20717648e+00 -1.66555928e+00]
 [-5.27032310e-01 1.20201284e-01 -5.27032310e-01 ... -1.02661853e+00
 -1.53624235e+00 -1.40998273e+00]
 [-3.44785986e-01 -3.30173390e-01 -3.44785986e-01 ... -5.62968376e-01
 -1.40904148e+00 -1.73122715e+00]]

[[ 1.36328735 0.21671014 1.36328735 ... -0.54951561 0.52977141
 0.95601528]
 [-0.83173827 -0.45885187 -0.83173827 ... 1.48295822 1.30400624
 1.16631813]
 [ 0.19918454 0.50623672 0.19918454 ... 1.24518065 0.93161341
 0.42880951]
 ...
 [ 0.77478605 0.18454052 0.77478605 ... 0.48697626 0.81902367
 0.5879681 ]
 [-0.30467295 0.97092687 -0.30467295 ... 0.40541613 0.41240667
 0.3090921 ]
 [-1.84899554 0.6349152 -1.84899554 ... -1.26284592 -0.37152924
 0.22890732]]
```

```
# CONVERT TO DATAFRAME
df_X_train_scaled = pd.DataFrame(X_train_scaled,columns=X_train.columns)
print(X_train_scaled.shape)
df_X_train_scaled.head()
```

(1643, 21)

|   | st_tmag   | st_logg   | TESS Mag  | ratio_eqt_teff | ratio_pl_orbper | ratio_pl_eqt | st_mass_est | log_pl_rade | log_pl_eqt | log_pl_orbper | ... | log_st_teff | log_st_rad | log_P |
|---|-----------|-----------|-----------|----------------|-----------------|--------------|-------------|-------------|------------|---------------|-----|-------------|------------|-------|
| 0 | -0.364401 | -1.842146 | -0.364401 | -1.457009      | -0.585745       | 2.702557     | -0.069912   | 0.989846    | -1.399313  | 1.398543      | ... | 0.547504    | 1.447906   | (     |
| 1 | -1.186924 | 0.068312  | -1.186924 | 0.434080       | 0.365515        | -0.583208    | 4.089414    | 0.332607    | 1.385515   | -0.726442     | ... | 2.233016    | 3.062754   | ,     |
| 2 | -0.714568 | 0.377558  | -0.714568 | 1.818849       | -0.287671       | -0.952187    | -0.114504   | -1.658112   | 1.164713   | -1.260956     | ... | -0.068423   | -0.293521  | .     |
| 3 | 0.389147  | -1.488280 | 0.389147  | 0.788600       | 0.150759        | -0.536471    | -0.030340   | 0.219393    | 1.072211   | -0.633990     | ... | 0.942312    | 1.295290   | ,     |
| 4 | -1.457820 | 0.795763  | -1.457820 | -0.655287      | -0.575239       | -0.266944    | -0.209496   | -0.566967   | -0.725599  | 0.121602      | ... | -0.730792   | -0.855515  | )     |

5 rows × 21 columns

```
# CONVERT TO DATAFRAME
df_X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_train.columns)
print(X_test_scaled.shape)
df_X_test_scaled.head()
```

(411, 21)

|   | st_tmag   | st_logg   | TESS Mag  | ratio_eqt_teff | ratio_pl_orbper | ratio_pl_eqt | st_mass_est | log_pl_rade | log_pl_eqt | log_pl_orbper | ... | log_st_teff | log_st_rad | log_P |
|---|-----------|-----------|-----------|----------------|-----------------|--------------|-------------|-------------|------------|---------------|-----|-------------|------------|-------|
| 0 | 1.363287  | 0.216710  | 1.363287  | -1.059647      | -0.495531       | 0.639758     | -0.139174   | 0.326978    | -0.857249  | 0.497766      | ... | 0.176164    | -0.278405  | (     |
| 1 | -0.831738 | -0.458852 | -0.831738 | -0.185482      | -0.500164       | 0.957114     | -0.242566   | 1.028647    | -0.262202  | 1.052395      | ... | -0.686008   | -0.339304  | ,     |
| 2 | 0.199185  | 0.506237  | 0.199185  | 0.004339       | 0.415156        | 0.047403     | -0.158487   | 0.516342    | 0.137896   | -0.654379     | ... | -0.079162   | -0.513047  | .     |
| 3 | -0.345611 | -0.426682 | -0.345611 | 1.787929       | 1.169922        | 0.107991     | -0.106026   | 1.529964    | 1.318307   | -0.456160     | ... | 0.339299    | 0.248497   | ,     |
| 4 | -1.977051 | -0.458852 | -1.977051 | 0.226139       | -0.211913       | -0.426912    | -0.123014   | 0.011392    | 0.461993   | -0.363889     | ... | 0.316613    | 0.209929   | )     |

5 rows × 21 columns

```
# CONVERT SERIES TO NUMPY ARRAY BEFORE SHAPING

scaler_y = StandardScaler()

y_train_array = y_train.to_numpy().reshape(-1, 1)
y_test_array = y_test.to_numpy().reshape(-1, 1)

# SCALE AND FLATTEN
y_train_scaled = scaler_y.fit_transform(y_train_array).ravel()
y_test_scaled = scaler_y.transform(y_test_array).ravel()

print(y_train_scaled.shape)
print(y_test_scaled.shape)
```

Show hidden output

```
# CONVERT TO DATAFRAME

df_y_train_scaled = pd.DataFrame(y_train_scaled)
df_y_train_scaled.columns = [y_cols]
df_y_train_scaled.head()
```

tfpwpwg\_disp\_encoded

|   |           |
|---|-----------|
| 0 | -1.017809 |
| 1 | -1.017809 |
| 2 | -1.017809 |
| 3 | -1.017809 |
| 4 | 0.982502  |

```
# CONVERT TO DATAFRAME
df_y_test_scaled = pd.DataFrame(y_test_scaled)
df_y_test_scaled.columns = [y_cols]
```

## 2.2 - APPLY PEARSON CORRELATION

```
# SAVE FEATURE NAMES FOR X_TRAIN/TEST
X_feature_names = X_train.columns
X_feature_names
```

```
Index(['st_tmag', 'st_logg', 'TESS Mag', 'ratio_eqt_teff', 'ratio_pl_orbper',
      'ratio_pl_eqt', 'st_mass_est', 'log_pl_rade', 'log_pl_eqt',
      'log_pl_orbper', 'log_TSM', 'log_st_teff', 'log_st_rad',
      'log_Predicted Mass (M_Earth)', 'log_pl_trandurh', 'log_pl_trandep',
      'log_pl_insol', 'log_st_dist', 'log_ESM', 'log_ratio_pl_st_rad',
      'log_ratio_rade_eqt'],
      dtype='object')
```

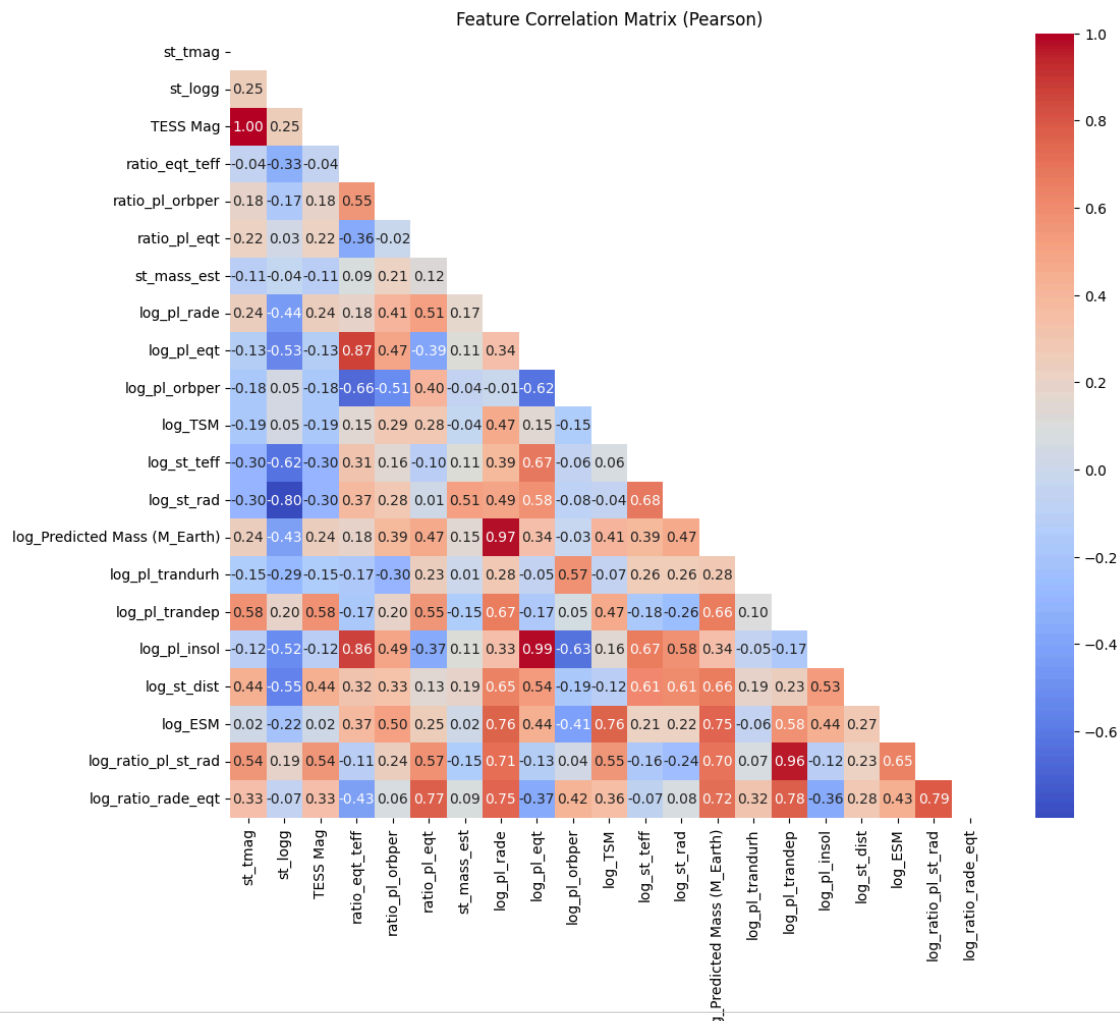
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

feature_names = X_feature_names
X_df = pd.DataFrame(X_train_scaled, columns=feature_names)

# Compute Pearson correlation
corr_matrix = X_df.corr()

# Mask upper triangle for cleaner heatmap
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

# Plot
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, mask=mask, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Feature Correlation Matrix (Pearson)")
plt.show()
```



```
# IDENTIFY PAIRS WITH CORRELATION ABOVE THRESHOLD
```

```
# THIS HELPS FLAG REDUNDANT FEATURES THAT MAY DILUTE CLASS SIGNAL OR INFLATE
# MODEL COMPLEXITY.

# SETTING THRESHOLD
# > 0.85 → CANDIDATES FOR REMOVAL OR CONSOLIDATION

threshold = 0.85
corr_pairs = corr_matrix.abs().stack().reset_index()
corr_pairs.columns = ['Feature A', 'Feature B', 'Correlation']
filtered = corr_pairs[(corr_pairs['Feature A'] != corr_pairs['Feature B']) &
                     (corr_pairs['Correlation'] > threshold)]
filtered
```

|     | Feature A                    | Feature B                    | Correlation |
|-----|------------------------------|------------------------------|-------------|
| 2   | st_tmag                      | TESS Mag                     | 1.000000    |
| 42  | TESS Mag                     | st_tmag                      | 1.000000    |
| 71  | ratio_eqt_teff               | log_pl_eqt                   | 0.867080    |
| 79  | ratio_eqt_teff               | log_pl_insol                 | 0.861143    |
| 160 | log_pl_rade                  | log_Predicted Mass (M_Earth) | 0.972464    |
| 171 | log_pl_eqt                   | ratio_eqt_teff               | 0.867080    |
| 184 | log_pl_eqt                   | log_pl_insol                 | 0.985321    |
| 280 | log_Predicted Mass (M_Earth) | log_pl_rade                  | 0.972464    |
| 334 | log_pl_trandep               | log_ratio_pl_st_rad          | 0.963284    |
| 339 | log_pl_insol                 | ratio_eqt_teff               | 0.861143    |
| 344 | log_pl_insol                 | log_pl_eqt                   | 0.985321    |
| 414 | log_ratio_pl_st_rad          | log_pl_trandep               | 0.963284    |

## 2.3 - APPLY SHAPELY ADDITIVE EXPLANATIONS (SHAP) TO DETERMINE FEATURE IMPORTANCE

Ranks features using SHAP from a class-weighted model trained on imbalanced, scaled data.

```
# SCALE AND TRAIN FOR SHAP ANALYSIS

from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

# TRAIN RF WITH CLASS WEIGHT ON class_weight on imbalanced data
rf_shap = RandomForestClassifier(class_weight='balanced', random_state=42)
rf_shap.fit(X_train_scaled, y_train)
```

```
▼ RandomForestClassifier ⓘ ?
RandomForestClassifier(class_weight='balanced', random_state=42)
```

### INTERPRETATION

SHAP returns a 3D array:

- 1643 samples:
- 22 features
- 2 outputs (classes)

**\*\*Shape: (n\_samples, n\_features, n\_classes)\*\***

For each sample and each feature, SHAP is providing a pair of attribution scores — one for class 0, one for class 1.

```
# COMPUTE SHAP IMPORTANCE (LONG EXECUTION)

import shap
import pandas as pd

explainer = shap.TreeExplainer(rf_shap)
shap_values = explainer.shap_values(X_train_scaled)

# SHAPE: (n_samples, n_features, n_classes)
shap_vals_class1 = shap_values[:, :, 1] # Class 1 attribution

# CREATE IMPORTANCE DICTIONARY
shap_df = pd.DataFrame(shap_vals_class1, columns=X_train.columns)
```

### 2.3.1 - RETAIN FEATURES CONTRIBUTING TO 95% OF THE TOTAL SHAP MAGNITUDE

```
# AGGREGATE SHAP VALUES (GLOBAL IMPORTANCE)

# SHAP: (n_samples, n_features, n_classes)

import numpy as np
import pandas as pd

# TO OBTAIN A SINGLE IMPORTANCE SCORE PER FEATURE, COMPUTE THE MEAN ABSOLUTE SHAP VALUE ACROSS BOTH SAMPLES AND CLASSES

mean_shap = np.abs(shap_values).mean(axis=(0, 2)) # Shape becomes (n_features,)
feature_importance = pd.Series(mean_shap, index=feature_names).sort_values(ascending=False)

# KEEP FEATURES CONTRIBUTING TO 95% OF THE TOTAL SHAP MAGNITUDE

cumulative_importance = feature_importance.cumsum() / feature_importance.sum()
selected_features = cumulative_importance[cumulative_importance <= 0.95].index.tolist()

print(f"Selected Features: {selected_features}")
```

```
Selected Features: ['log_pl_trandep', 'log_pl_eqt', 'ratio_pl_eqt', 'log_st_dist', 'log_ratio_rade_eqt', 'st_mass_est', 'ratio_pl_orbper', 'ratio_eqt_teff', 'log_pl_in
```

## 3 - APPLY PCA

### 3.1 - FULL FEATURE SET

```
# FIT PCA ON FULL TRAINING DATA

from sklearn.decomposition import PCA

# SET THE NUMBER OF COMPONENTS OR USE EXPLAINED VARIANCE THRESHOLD
pca = PCA() # pca = PCA(n_components=0.95) # Keep components explaining 95% of variance

X_train_scaled_pca = pca.fit_transform(X_train_scaled)
```

```
# APPLY TO TEST DATA

# THIS ENSURES THE SAME TRANSFORMATION IS APPLIED CONSISTENTLY ACROSS TRAIN/TEST SPLITS
X_test_scaled_pca = pca.transform(X_test_scaled)
```

```
# FIT PCA ON FULL TRAINING DATA

from sklearn.decomposition import PCA

# SET THE NUMBER OF COMPONENTS OR USE EXPLAINED VARIANCE THRESHOLD
pca = PCA() # pca = PCA(n_components=0.95) # Keep components explaining 95% of variance

X_train_scaled_pca = pca.fit_transform(X_train_scaled)
```

```
df_y_train = pd.DataFrame(y_train)
```

```
# PLOT PCA1 AND PCA2 COMPONENTS

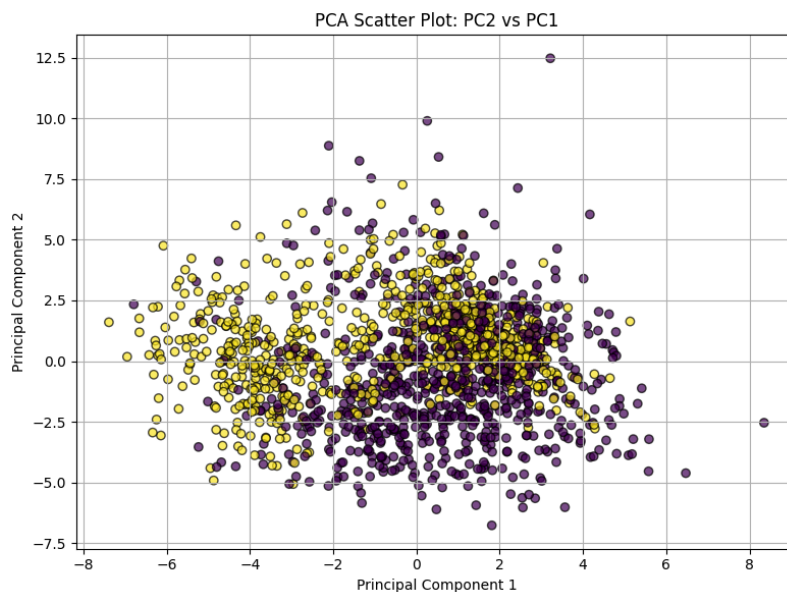
# PRODUCE A SCATTER PLOT OF PC2 VS PC1 TO VISUALISE HOW SHAP-SELECTED FEATURES CLUSTER IN PCA SPACE.

import matplotlib.pyplot as plt

# ASSIGN TO SERIES
y_train_series = df_y_train["tfopwg_disp_encoded"]

# EXTRACT PC1 AND PC2
pc1 = X_train_scaled_pca[:, 0]
pc2 = X_train_scaled_pca[:, 1]

# COLOUR BY CLASS LABEL
plt.figure(figsize=(8, 6))
plt.scatter(pc1, pc2, c=y_train_series, cmap='viridis', edgecolor='k', alpha=0.7)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA Scatter Plot: PC2 vs PC1')
plt.grid(True)
plt.tight_layout()
plt.show()
```



#### INTERPRETATION

The above PCA plot shows data points clearly separated into two main clusters, implying that data has two distinct groups that are separated along the first principal component (PC1).

The x-axis represents PC1 with the majority of the variance in the data is captured by PC1 with two groups appearing clearly distinct along this axis.

PC2, represented by the y-axis, contributes to some variance due to less data spread and is less effective in distinguishing between the groups compared to PC1.

The data appears to have a strong underlying structure where the two classes are clearly distinguishable based on the first principal component.

3 - APPLY SYNTHETIC MINORITY OVER-SAMPLING TECHNIQUE (SMOTE)

Addresses class imbalance by generating synthetic samples for the minority class.  
Creates new synthetic sample by interpolating between existing minority class samples. Selects two or more similar instances from the minority class and generates new instances that lie between them in feature space  
Minority class - 'CONFIRMED' with fewer entries than 'FALSE POSITIVE'.

3.1 - APPLY SMOTE

```
# APPLY SMOTE

import xgboost as xgb
import shap
import pandas as pd

from imblearn.over_sampling import SMOTE
from collections import Counter

print("Original dataset dimension: ", X_train.shape)
print()

# APPLY SMOTE TO SCALED DATA
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(df_X_train_scaled, df_y_train)
# X_train_resampled, y_train_resampled = smote.fit_resample(X_train_scaled, y_train)

print('APPLYING SMOTE: \n')
print('X (train): ')
print("Feature data dimension: ", X_train_resampled.shape, '\n')

print('y (train): ')
print("Target data dimension: ", y_train_resampled.shape, '\n')
```

Original dataset dimension: (1643, 21)

APPLYING SMOTE:

X (train):  
Feature data dimension: (1672, 21)

y (train):  
Target data dimension: (1672, 1)

X\_train\_resampled

|      | st_tmag   | st_logg   | TESS Mag  | ratio_eqt_teff | ratio_pl_orbper | ratio_pl_eqt | st_mass_est | log_pl_rade | log_pl_eqt | log_pl_orbper | ... | log_st_teff | log_st_rad | log_P |
|------|-----------|-----------|-----------|----------------|-----------------|--------------|-------------|-------------|------------|---------------|-----|-------------|------------|-------|
| 0    | -0.364401 | -1.842146 | -0.364401 | -1.457009      | -0.585745       | 2.702557     | -0.069912   | 0.989846    | -1.399313  | 1.398543      | ... | 0.547504    | 1.447906   |       |
| 1    | -1.186924 | 0.068312  | -1.186924 | 0.434080       | 0.365515        | -0.583208    | 4.089414    | 0.332607    | 1.385515   | -0.726442     | ... | 2.233016    | 3.062754   |       |
| 2    | -0.714568 | 0.377558  | -0.714568 | 1.818849       | -0.287671       | -0.952187    | -0.114504   | -1.658112   | 1.164713   | -1.260956     | ... | -0.068423   | -0.293521  |       |
| 3    | 0.389147  | -1.488280 | 0.389147  | 0.788600       | 0.150759        | -0.536471    | -0.030340   | 0.219393    | 1.072211   | -0.633990     | ... | 0.942312    | 1.295290   |       |
| 4    | -1.457820 | 0.795763  | -1.457820 | -0.655287      | -0.575239       | -0.266944    | -0.209496   | -0.566967   | -0.725599  | 0.121602      | ... | -0.730792   | -0.855515  |       |
| ...  | ...       | ...       | ...       | ...            | ...             | ...          | ...         | ...         | ...        | ...           | ... | ...         | ...        |       |
| 1667 | -1.243042 | -0.146889 | -1.243042 | 3.922546       | 0.456229        | -0.923962    | 0.183174    | -0.353708   | 2.731885   | -1.099252     | ... | 1.993422    | 0.767436   |       |
| 1668 | 1.005274  | 0.522833  | 1.005274  | 0.475719       | -0.116952       | -0.388102    | 0.006283    | 0.496756    | 0.991458   | -0.137081     | ... | 1.204763    | -0.062529  |       |
| 1669 | -1.266951 | -0.170195 | -1.266951 | 1.099283       | 0.036161        | -0.907424    | 0.054824    | -1.090769   | 1.496363   | -1.221605     | ... | 1.567631    | 0.514457   |       |
| 1670 | -0.745833 | 0.356014  | -0.745833 | 3.200194       | 0.717999        | -0.657176    | -0.161478   | 0.312084    | 1.659025   | -0.926552     | ... | -0.099889   | -0.434964  |       |
| 1671 | -0.189026 | -0.197132 | -0.189026 | -1.522083      | -0.751223       | 1.354926     | -0.147613   | 0.118769    | -1.739182  | 2.499082      | ... | 0.110206    | -0.050170  |       |

1672 rows × 21 columns

```
# RECONSTRUCT DATAFRAME WITH ORIGINAL COLUMN NAMES
df_X_train_resampled = pd.DataFrame(X_train_resampled)
df_X_train_resampled.columns = [X_train_resampled.columns]
print(df_X_train_resampled.shape)
df_X_train_resampled.head()
```

(1672, 21)

|   | st_tmag   | st_logg   | TESS Mag  | ratio_eqt_teff | ratio_pl_orbper | ratio_pl_eqt | st_mass_est | log_pl_rade | log_pl_eqt | log_pl_orbper | ... | log_st_teff | log_st_rad | log_P      |
|---|-----------|-----------|-----------|----------------|-----------------|--------------|-------------|-------------|------------|---------------|-----|-------------|------------|------------|
| 0 | -0.364401 | -1.842146 | -0.364401 | -1.457009      | -0.585745       | 2.702557     | -0.069912   | 0.989846    | -1.399313  | 1.398543      | ... | 0.547504    | 1.447906   | Mass (M_Ea |
| 1 | -1.186924 | 0.068312  | -1.186924 | 0.434080       | 0.365515        | -0.583208    | 4.089414    | 0.332607    | 1.385515   | -0.726442     | ... | 2.233016    | 3.062754   |            |
| 2 | -0.714568 | 0.377558  | -0.714568 | 1.818849       | -0.287671       | -0.952187    | -0.114504   | -1.658112   | 1.164713   | -1.260956     | ... | -0.068423   | -0.293521  |            |
| 3 | 0.389147  | -1.488280 | 0.389147  | 0.788600       | 0.150759        | -0.536471    | -0.030340   | 0.219393    | 1.072211   | -0.633990     | ... | 0.942312    | 1.295290   |            |
| 4 | -1.457820 | 0.795763  | -1.457820 | -0.655287      | -0.575239       | -0.266944    | -0.209496   | -0.566967   | -0.725599  | 0.121602      | ... | -0.730792   | -0.855515  |            |

5 rows × 21 columns

```
# RECONSTRUCT DATAFRAME WITH ORIGINAL COLUMN NAMES
df_y_train_resampled = pd.DataFrame(y_train_resampled)
```



```
df_y_train_resampled.columns = [y_cols]
df_y_train_resampled.shape
```

```
(1672, 1)
```

### 3.2 - PCA SCATTER PLOT - FULL SET

```
# INSPECT FEATURE SEPARABILITY

# REDUCE TO 2D AND COLOUR BY CLASS

from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

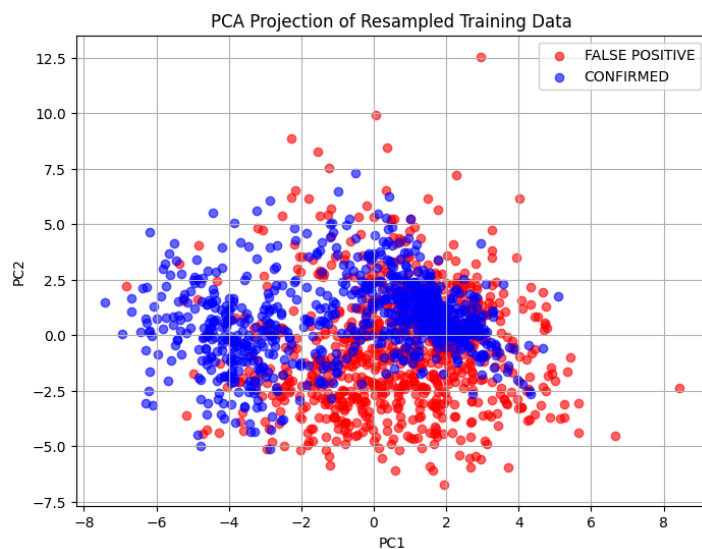
X_pca = PCA(n_components=2).fit_transform(X_train_resampled)

# CONVERT LABELS TO NUMPY ARRAY IF NEEDED
# ENSURE LABELS ARE 1-D ARRAY
y_labels = np.array(y_train_resampled).ravel()

# CREATE CLASS MASKS
false_pos_mask = (y_labels == 0)
confirmed_mask = (y_labels == 1)

# PLOT
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[false_pos_mask, 0], X_pca[false_pos_mask, 1],
            c='red', label='FALSE POSITIVE', alpha=0.6)
plt.scatter(X_pca[confirmed_mask, 0], X_pca[confirmed_mask, 1],
            c='blue', label='CONFIRMED', alpha=0.6)

plt.title("PCA Projection of Resampled Training Data")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.legend(loc='best')
plt.grid(True)
plt.show()
```



#### INTERPRETATION

The second PCA graph, after applying SMOTE, shows a more balanced and distinct separation between the two classes, compared to the first PCA graph. By generating synthetic points for the minority class, there is a more even distribution of data points and less overlap across both classes. SMOTE improves data separability and the second PCA graph shows more uniform density, which may help to increase model performance by reducing underfitting on the minority class. Therefore, applying SMOTE before PCA helped create a clearer and more balanced representation of the data in the feature space.

#### FEATURE IMPORTANCE

```
# FEATURE IMPORTANCE
# PICKUP HIGHLY CORRELATED (BEST) FEATURES OF CLASSIFICATION DATA

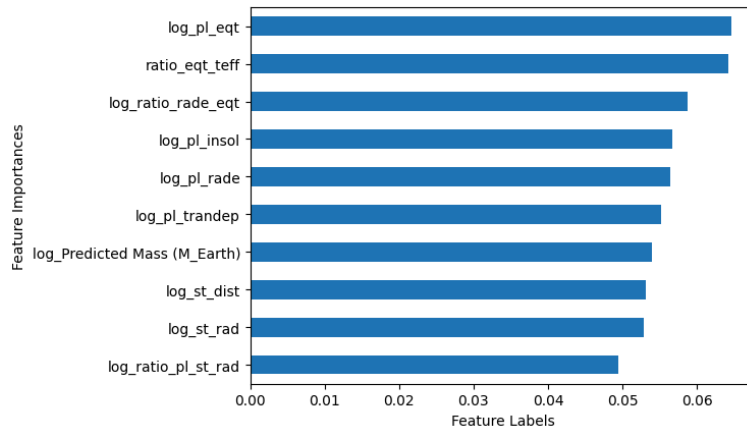
# EXTRA TREE CLASSIFIER IS USED TO EXTRACT THE TOP 10 FEATURES FOR THE DATASET
# FEATURE IMPORTANCE IS AN INBUILT CLASS THAT COMES WITH TREE-BASED CLASSIFIERS

from sklearn.ensemble import ExtraTreesClassifier
import matplotlib.pyplot as plt

model = ExtraTreesClassifier()
model.fit(X,y)
print(model.feature_importances_)
print()

# PLOT THE GRAPH OF FEATURE IMPORTANCE
feat_importances = pd.Series(model.feature_importances_, index=X.columns)
feat_importances.nlargest(10).sort_values().plot(kind='barh')
plt.xlabel('Feature Labels')
plt.ylabel('Feature Importances')
# plt.title('Comparison of different Feature Importances')
plt.show()
```

```
[0.03330495 0.03946974 0.03438066 0.064278 0.0394753 0.04744901
0.03486823 0.0564621 0.06466943 0.0462003 0.03031472 0.04313229
0.0529154 0.05397961 0.04820479 0.05522416 0.0566705 0.05313164
0.03759627 0.04945442 0.05881848]
```



#### INTERPRETATION

The most important features for the model are primarily related to an planet's temperature and physical characteristics (e.g radius).Features such as insolation (log\_pl\_insol), equilibrium temperature (log\_pl\_eqt) and effective temperature ratio (ratio\_eq\_teff) are primary factors in model decision making.

Stellar properties such as radius (log\_st\_rad) and distance (log\_st\_dist) suggest that while they have some influence, they are not predictive in the outcome in the modelling especially after SMOTE. This could have implications for how the model performs and which features to focus on in further analysis or when improving the model

## 4.MODELLING

#### GENERATE MACHINE LEARNING MODELS FOR:

##### Support Vector Classifier/Machine: SVC (SVM)

- Well-suited for binary classification tasks where feature engineering is well-optimised

##### Random Forest (RF)

- Suitable for handling noisy astronomical data, offering strong interpretability, valuable feature importance insights and robustness in finding exoplanets with pronounced transit signals
- Used in Autovetter for KEPLER and adapted for TESS

##### Light Gradient Boosting Machine (LIGHTGBM)

- Excellent for tabular data with mixed feature types
- Integrates well with SHAP for interpretability

##### Neural Networks - Multi-Layer Perceptron(MLP)

- MLPs are flexible and model either nonlinear relationships on tabular data
- Captures abstract patterns and feature interactions
- Works well with TSFRESH or PCA inputs

#### DECLARATIONS

```
# REPORTING METRICS
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, balanced_accuracy_score, classification_report, cohen_kappa_score, f1_score
from sklearn.metrics import accuracy_score
```

```
from sklearn.metrics import roc_curve, auc
from sklearn.metrics import mean_squared_error,r2_score # RMSE, Chi-square
```

```
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import make_pipeline # library to make a pipeline
from sklearn.pipeline import Pipeline
```

```
import pandas as pd
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from scipy.stats import randint
```

```
import matplotlib.pyplot as plt
```

## OPTIMISED MODELS - HYPERPARAMETER TUNING

```
# SKLEARN Library
from sklearn import datasets
from sklearn.model_selection import cross_val_predict
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import RepeatedKFold

# ANN - Multilayer Perceptron model
# Split method
# Best parameter value

from sklearn.model_selection import cross_val_score
from sklearn.metrics import accuracy_score

# Cross validation
# Reporting Metrics
```

```

from sklearn.metrics import mean_squared_error          # RMSE
from sklearn.neural_network import MLPClassifier        # ANN Model
from sklearn.preprocessing import StandardScaler        # Standardises features by removing the mean and scaling to unit variance
from sklearn.pipeline import Pipeline                  # Pipeline Library

```

```

# SKLEARN LIBRARY
from sklearn import datasets
from sklearn.model_selection import cross_val_predict  # ANN - Multilayer Perceptron model
from sklearn.model_selection import train_test_split   # Split method
from sklearn.model_selection import GridSearchCV       # Best parameter value
from sklearn.model_selection import cross_val_score    # Cross validation
from sklearn.metrics import classification_report, confusion_matrix # Reporting Metrics
from sklearn.metrics import accuracy_score            # Reporting Metrics

```

```

from sklearn.pipeline import make_pipeline # library to make a pipeline
from sklearn.pipeline import Pipeline

```

## 4.1 - RANDOM FOREST

### 4.1.1 - All Features

```

# INSTANTIATE THE RANDOM FOREST MODEL(RF)

# FIT PR TO THE SCALED RESAMPLED TRAINING DATASET (X_train_resampled)
# PREDICT WITH SCALED TESTING DATASET (X_test_scaled)

print('Random Forest - Hypertuning of Parameters')

# SET UP THE RANDOM FOREST CLASSIFIER AND HYPERPARAMETER TUNUNG
# CREATE A PIPELINE
pca=PCA()
predictor = RandomForestClassifier(random_state=42)

# Make pipeline
pipe_RF_hyper = make_pipeline(pca, predictor) # Build model with PCA & Rfc only

```

Random Forest - Hypertuning of Parameters

```

# FIT MODEL WITH THE TRAINING SUBSET
pipe_RF_hyper.fit(X_train_resampled,y_train_resampled) # scaled data (x_sc)

# PREDICT USING TEST SUBSET
y_RF_hyper_predict_test = pipe_RF_hyper.predict(X_test_scaled)

# PREDICT USING TRAIN SUBSET
y_RF_hyper_predict_train = pipe_RF_hyper.predict(X_train_resampled)

```

```

# DETERMINE PARAMETERS FOR RANDOMSEARCHRV
sorted(pipe_RF_hyper.get_params().keys())

```

```

['memory',
'pca',
'pca__copy',
'pca__iterated_power',
'pca__n_components',
'pca__n_oversamples',
'pca__power_iteration_normalizer',
'pca__random_state',
'pca__svd_solver',
'pca__tol',
'pca__whiten',
'randomforestclassifier',
'randomforestclassifier__bootstrap',
'randomforestclassifier__ccp_alpha',
'randomforestclassifier__class_weight',
'randomforestclassifier__criterion',
'randomforestclassifier__max_depth',
'randomforestclassifier__max_features',
'randomforestclassifier__max_leaf_nodes',
'randomforestclassifier__max_samples',
'randomforestclassifier__min_impurity_decrease',
'randomforestclassifier__min_samples_leaf',
'randomforestclassifier__min_samples_split',
'randomforestclassifier__min_weight_fraction_leaf',
'randomforestclassifier__monotonic_cst',
'randomforestclassifier__n_estimators',
'randomforestclassifier__n_jobs',
'randomforestclassifier__oob_score',
'randomforestclassifier__random_state',
'randomforestclassifier__verbose',
'randomforestclassifier__warm_start',
'steps',
'transform_input',
'verbose']

```

```

# SET UP PARAMETERS FOR RANDOMRIDSEARCHCV

import numpy as np    # Operations on arrays

# DEFAULT CRITERION IS GINI

params_grid = {
    'randomforestclassifier__n_estimators': [300, 500, 700, 900], # NO. of trees in the RF
    'randomforestclassifier__max_depth': [None, 4, 6, 8],
    'randomforestclassifier__max_features': ['sqrt', 'log2', 0.3, 0.5, 1],
    'randomforestclassifier__min_samples_leaf': [1, 2, 5, 10, 0.05, 0.1],
    'randomforestclassifier__bootstrap': [True, False],
    'randomforestclassifier__min_samples_split': [2, 5, 10, 13],
    'randomforestclassifier__max_leaf_nodes': [None, 10, 20, 50]
}

scores = ["f1", "balanced_accuracy"]

```

```

# SEARCH FOR OPTIMAL MODEL FROM PARAM GRID

```

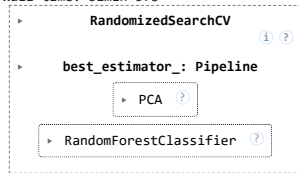
```

grid_RF_opt = RandomizedSearchCV(
    estimator=pipe_RF_hyper,
    param_distributions=params_grid,
    n_iter=100,
    cv=5,
    verbose=1,
    # random_state=42,
    n_jobs=-1,
    return_train_score=True
)

# CHECKING HOW MUCH TIME TAKEN TO FIND OPTIMUM VALUE
%time grid_RF_opt.fit(X_train_resampled,y_train_resampled) # scaled data (x_sc)

```

Fitting 5 folds for each of 100 candidates, totalling 500 fits  
CPU times: user 40.6 s, sys: 2.32 s, total: 42.9 s  
Wall time: 31min 57s



```

# PRINT THE BEST PARAMETERS AND THE BEST SCORE
print("Best parameters found: ", grid_RF_opt.best_params_)
print("Best accuracy score: ", grid_RF_opt.best_score_)

```

```

Best parameters found: {'randomforestclassifier__n_estimators': 700, 'randomforestclassifier__min_samples_split': 2, 'randomforestclassifier__min_samples_leaf': 2, 'randomforestclassifier__max_depth': None, 'randomforestclassifier__max_leaf_nodes': None, 'randomforestclassifier__max_features': 0.3, 'randomforestclassifier__bootstrap': False}
Best accuracy score: 0.8564590222539994

```

## INTERPRETATION

- **n\_estimators = 500**- The dataset has enough complexity or noise that the model stabilises only with a decent ensemble of trees
- **min\_samples\_split = 2 & min\_samples\_leaf = 2** - optimiser determined that limited splitting did not improve generalisation. Data benefits from fine-grained splits early in the tree. Preventing 1-sample leaves reduces overfitting on the noisy points. The data is fairly clean or well-separated aafter processing/resampling
- **max\_depth=None** - dataset has complex non-linear structure that benefits from deeper fully grown trees
- **max\_leaf\_nodes=None** - dataset benefits from flexible trees rather than eexplicit pruning
- **max\_features = 0.3** - the model samples only third of the available features at each split. There is strong feature redundancy giving better splits.
- **bootstrap = false** - full dataset is used for every tree, stabilising splits and reduces variance per tree, giving more consistent learning

High randomness and diversity produces high ensemble diversity and more stable model generalisation

A best accuracy (~0.856) is a strong score, especially for astrophysical classification like TESS candidates where noise is significant, classes overlap and resampled datasets can distort decision boundaries

```

# ASSIGN BEST PARAMETERS AND FIT MODEL
# CREATE RF MODEL WITH OPTIMAL PARAMETERS

from sklearn import metrics
from sklearn.model_selection import cross_val_score

# ASSIGN THE BEST PARAMETERS AND FIT THE MODEL
model_RF_opt = grid_RF_opt.best_estimator_

print(X_train_resampled.shape)

# FIT THE MODEL
model_RF_opt.fit(X_train_resampled, y_train_resampled) # scaled data (x_sc)

# PREDICTING TEST DATASET RESULT UNDER MODEL
y_predict_RF_opt = model_RF_opt.predict(X_test_scaled) # scaled data (x_sc)

# MEASURE THE PERFORMANCE
print('Accuracy score %.3f' %metrics.accuracy_score(y_test, y_predict_RF_opt)) # Scaled data (x_sc)

```

```

(1672, 21)
Accuracy score 0.859

```

```

# GENERATE CLASSIFIICATION REPORT

```

```

report = classification_report(y_test, y_predict_RF_opt, output_dict=True)
print(report)

```

```

{'0': {'precision': 0.8738317757009346, 'recall': 0.8577981651376146, 'f1-score': 0.8657407407407407, 'support': 218.0}, '1': {'precision': 0.8426395939086294, 'recall': 0.8577981651376146, 'f1-score': 0.8507407407407407, 'support': 218.0}}

```

```

# EVALUATE THE MODEL - CONFUSION MATRIX
conf_matrix = confusion_matrix(y_test, y_predict_RF_opt)
f1 = f1_score(y_test, y_predict_RF_opt, average='weighted')
report = classification_report(y_test, y_predict_RF_opt, output_dict=True)
accuracy = balanced_accuracy_score(y_test, y_predict_RF_opt)
kappa = cohen_kappa_score(y_test, y_predict_RF_opt)

print(f"F1 Score: {f1}")
print(f"Kappa Score: {kappa}")
print(f"Accuracy Score: {accuracy}")
print(f"Confusion Matrix:\n{conf_matrix}")
print(report)

```

```

F1 Score: 0.8589511371749814
Kappa Score: 0.7170496629000095
Accuracy Score: 0.8589508960403098

```

```
Confusion Matrix:
[[187  31]
 [ 27 166]]
{'0': {'precision': 0.8738317757009346, 'recall': 0.8577981651376146, 'f1-score': 0.8657407407407407, 'support': 218.0}, '1': {'precision': 0.8426395939086294, 'recall': 0.858881, 'f1-score': 0.8504705881158655, 'support': 193.0}}
```

```
# REPORTING METRICS
# EVALUATE THE MODEL

# CONVERT THE REPORT TO A DATAFRAME
df_report = pd.DataFrame(report).transpose()

# RENAME THE INDEX LABELS
df_report.rename(index={'0': 'FALSE POSITIVE', '1': 'CONFIRMED'}, inplace=True)

# DISPLAY THE DATAFRAME
print(df_report, '\n')
```

|                | precision | recall   | f1-score | support    |
|----------------|-----------|----------|----------|------------|
| FALSE POSITIVE | 0.873832  | 0.857798 | 0.865741 | 218.000000 |
| CONFIRMED      | 0.842640  | 0.860104 | 0.851282 | 193.000000 |
| accuracy       | 0.858881  | 0.858881 | 0.858881 | 0.858881   |
| macro avg      | 0.858236  | 0.858951 | 0.858511 | 411.000000 |
| weighted avg   | 0.859184  | 0.858881 | 0.858951 | 411.000000 |

# SAVE MODEL GRID

```
import joblib

# SAVE MODEL VALUES
joblib.dump(grid_RF_opt, ('/content/drive/MyDrive/grid_RF_opt_PORT.joblib'))

['/content/drive/MyDrive/grid_RF_opt_PORT.joblib']
```

```
# STYLE THE DATAFRAME
styled_report = df_report.style \
    .background_gradient(cmap='YlGnBu') \
    .set_caption('RF - Classification Report') \
    .format({'precision': '{:.2f}', 'recall': '{:.2f}', 'f1-score': '{:.2f}', 'support': '{:.0f}'}) \
    .set_properties(**{'text-align': 'center'}) \
    .set_table_styles([
        {'selector': 'caption',
         'props': 'caption-side: top; font-size: 16px; font-weight: bold;'
        }])

plt.figure(figsize=(16, 16))
display(styled_report)
```

|                | precision | recall | f1-score | support |
|----------------|-----------|--------|----------|---------|
| FALSE POSITIVE | 0.87      | 0.86   | 0.87     | 218     |
| CONFIRMED      | 0.84      | 0.86   | 0.85     | 193     |
| accuracy       | 0.86      | 0.86   | 0.86     | 1       |
| macro avg      | 0.86      | 0.86   | 0.86     | 411     |
| weighted avg   | 0.86      | 0.86   | 0.86     | 411     |

<Figure size 1600x1600 with 0 Axes>

```
# CREATE A CONFUSION MATRIX WITH CUSTOM LABELS
cm_RF_clf_opt = confusion_matrix(y_test, y_predict_RF_opt)

disp = ConfusionMatrixDisplay(confusion_matrix=cm_RF_clf_opt, display_labels=['FALSE POSITIVE', 'CONFIRMED'])

# PLOT THE CONFUSION MATRIX
disp.plot(cmap='Purples')
plt.show()
```



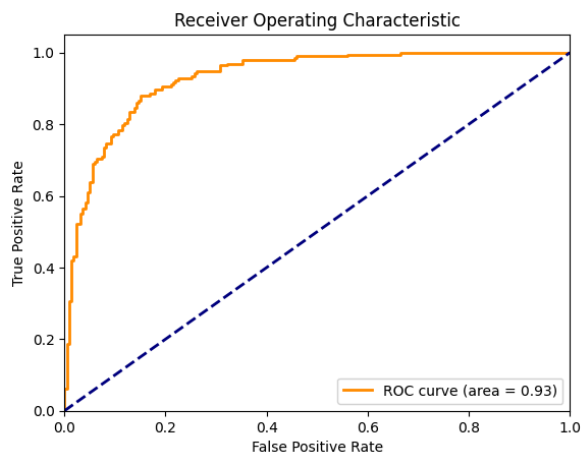
```
# ROC CURVE

# PREDICT PROBABILITIES
y_prob = model_RF_opt.predict_proba(X_test_scaled)[: , 1]

# COMPUTE ROC CURVE AND AREA
fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
```

```
# PLOT ROC CURVE
```

```
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc='lower right')
plt.show()
```



## APPLY SHAP ANALYSIS

```
import shap
import matplotlib.pyplot as plt

# 🌀 USE THE ORIGINAL SVM TRAINED ON SCALED FEATURES (BEFORE PCA)
best_RF = grid_RF_opt.best_estimator_.named_steps['randomforestclassifier']

# 🌀 USE KERNEL EXPLAINER (MODEL-AGNOSTIC) FOR SVM
explainer = shap.KernelExplainer(best_RF.predict, X_test_scaled[:100]) # use a sample subset for background
shap_values = explainer.shap_values(X_test_scaled[:100]) # explanation on test set subset

# 📊 SUMMARY PLOT: GLOBAL FEATURE IMPORTANCE
shap.summary_plot(shap_values, X_test_scaled[:100], feature_names= df_X_test_scaled.columns)
```

